WEEK 1 · DAY 5 · OVERVIEW

Case Study

Ridge, Lasso, and the Spread Cross-Section



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 4's log model misses a typical unseen bond by 33.1 bp, using only the rating and years to maturity.

The credit team asks to add every other column the file holds, sector, duration, coupon, amount and issuer, and let a penalty sort out which earn their place.

It wants two misses reported honestly: on a new bond of a known issuer, and on a bond of an issuer never seen.

The bonds and issuers are invented for this course, and every number below describes them, never a view on one.

OBJECTIVES

Build one pipeline that prepares every column on the training bonds only.

Fit least squares, ridge and lasso on the same 30 columns, each penalty chosen by cross-validation, read the columns lasso keeps against the recipe, and score issuers the model has never seen.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 5

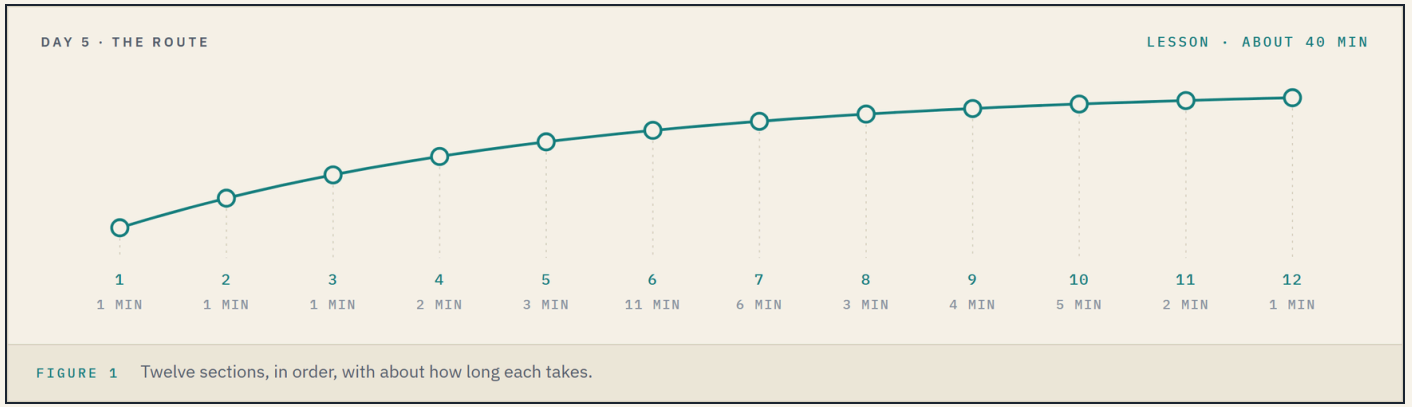


FIGURE 1 Twelve sections, in order, with about how long each takes.

	SECTION	ABOUT
1	Setting Up	1 min
2	Loading the Data	1 min
3	Data Overview	1 min
4	Exploratory Data Analysis	2 min
5	Data Preparation for Modeling	3 min
6	Model Building	11 min

	SECTION	ABOUT
7	Choosing Alpha	6 min
8	Model Performance Evaluation	3 min
9	Reading the Coefficients	4 min
10	The Split by Issuer	5 min
11	Where the Model Fails	2 min
12	Insights and Conclusion	1 min

Time. About 40 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, RidgeCV, LassoCV
```

Loading the Data

THE DATA DICTIONARY

- **`cusip`**: the bond's identifier
- **`issuer`** and **`ticker`**: the issuer's name and short code
- **`sector`**: one of 10 industry sectors
- **`rating`**: credit rating, AAA to CCC
- **`coupon`**: annual coupon, percent of par
- **`maturity`** and **`as_of_date`**: dates; years to maturity is built from them
- **`oas`**: option-adjusted spread, bp; **the target is its log**
- **`duration`**: modified duration, years
- **`amount_outstanding`**: face value outstanding, dollars

READING THE BONDS, THEN SPLITTING THEM

```
bonds = pd.read_csv(DATA + "benchmark.csv")
train, test = train_test_split(data, test_size=0.25, random_state=SEED)
print("Training bonds:", len(train), " Test bonds:", len(test))
```

```
Training bonds: 615  Test bonds: 206
```

From here the test bonds only score a fitted model; nothing is chosen on them.

Check the four numbers and their scales.

Data Overview

THE CANDIDATE COLUMNS

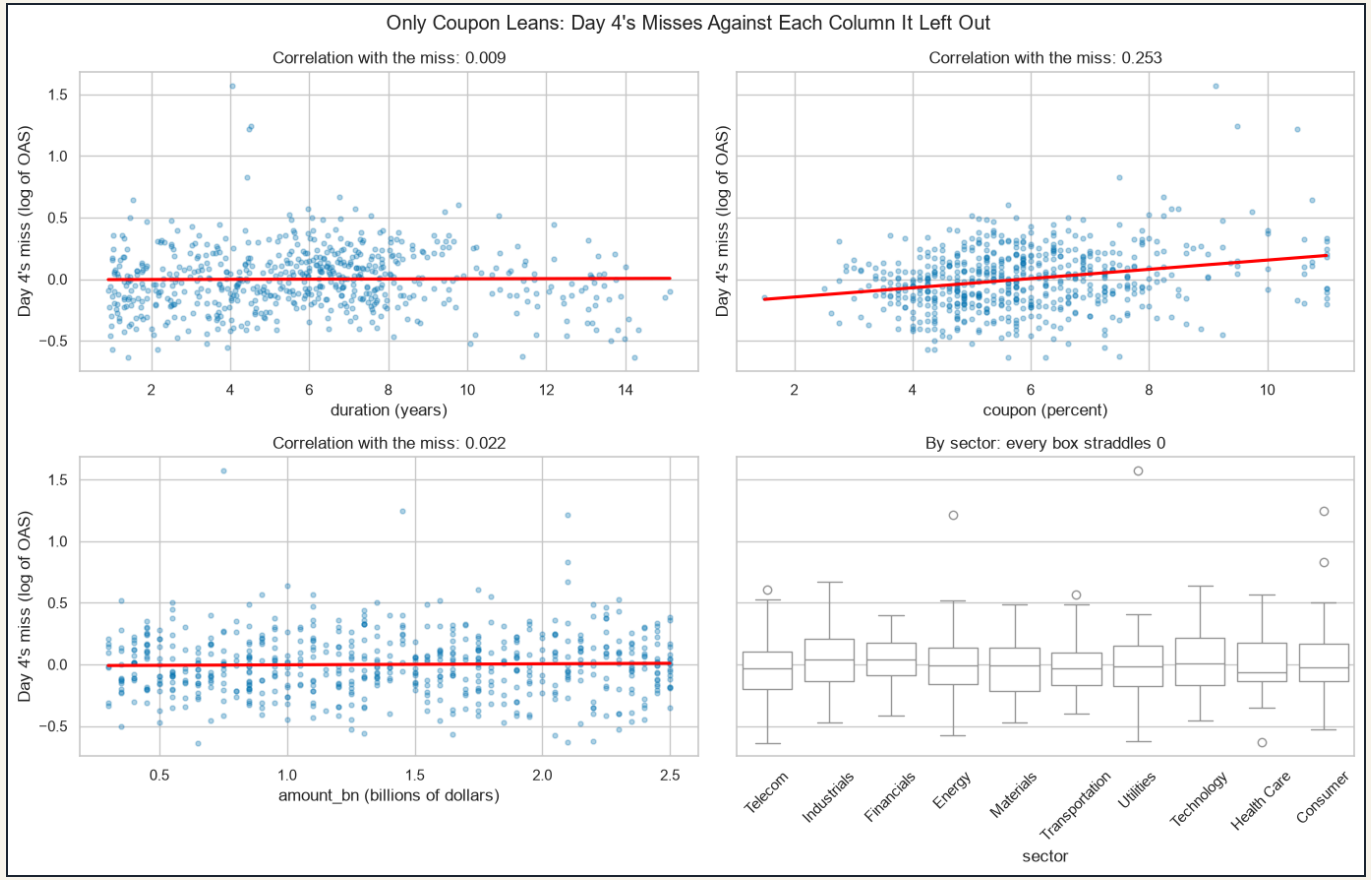
	TICKER	RATING	SECTOR
712	ELVX	A+	Telecom
181	HESN	A+	Industrials
778	PLEA	AA	Financials
716	PLER	BBB-	Energy

THEIR SCALES

	YEARS_TO_MATURITY	DURATION	COUPON	AMOUNT_BN
count	615.000	615.000	615.000	615.000
mean	8.763	5.918	5.873	1.394
std	6.792	3.224	1.698	0.657
min	0.964	0.915	1.500	0.300
25%	4.030	3.454	4.750	0.850

A coefficient's size depends on its column's unit.

Exploratory Data Analysis

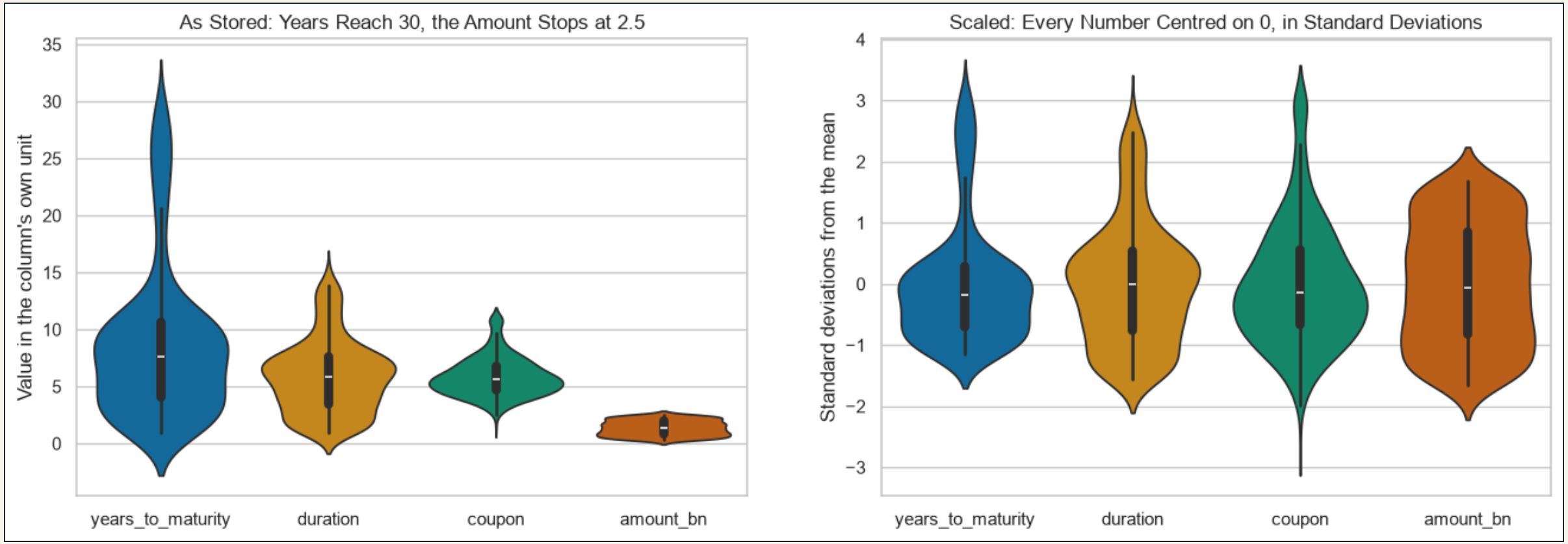


WHAT DAYS 1 TO 4 FOUND

- OAS has a long right tail, so the target is its log (Day 4).
- Each rating takes its own step, measured from A (Day 2).
- Sector columns fitted noise (Days 2 and 3).
- Duration and years to maturity are near-copies (Day 3).
- Coupon moves with spread: correlation 0.72 (Day 1).
- Issuer averages account for 42.8 percent of the log model's squared misses (Day 4).

Only coupon leans, and the issuer holds much of the rest. We give the model every column and let a penalty sort them out.

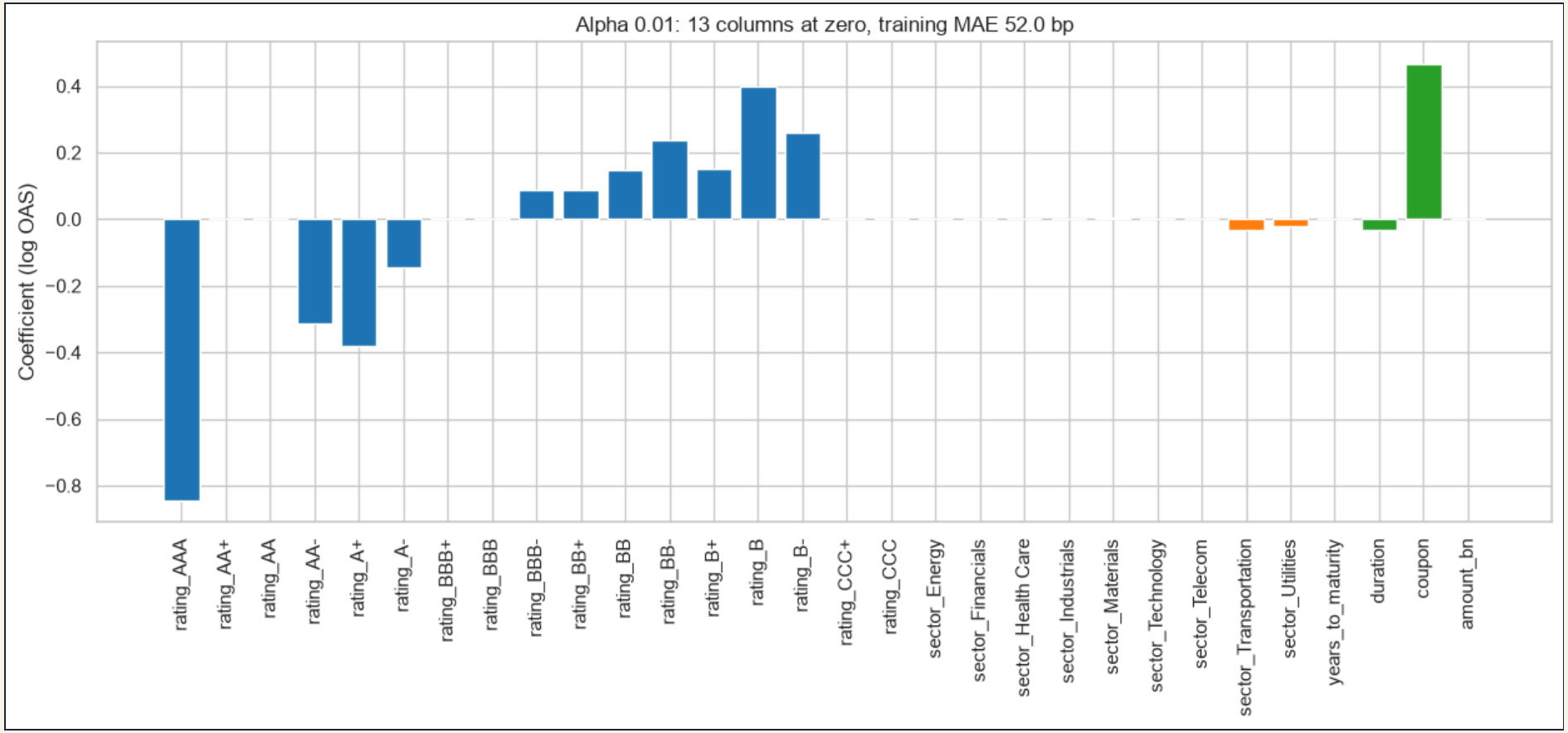
Data Preparation for Modeling



Every column now goes through one object that learns from the rows it is fitted on, and only those.

Fit three models and trace their coefficient paths.

Model Building



The training miss climbs from 34.5 to 52.0 bp: lasso gives up fit to drop columns.

Choosing Alpha

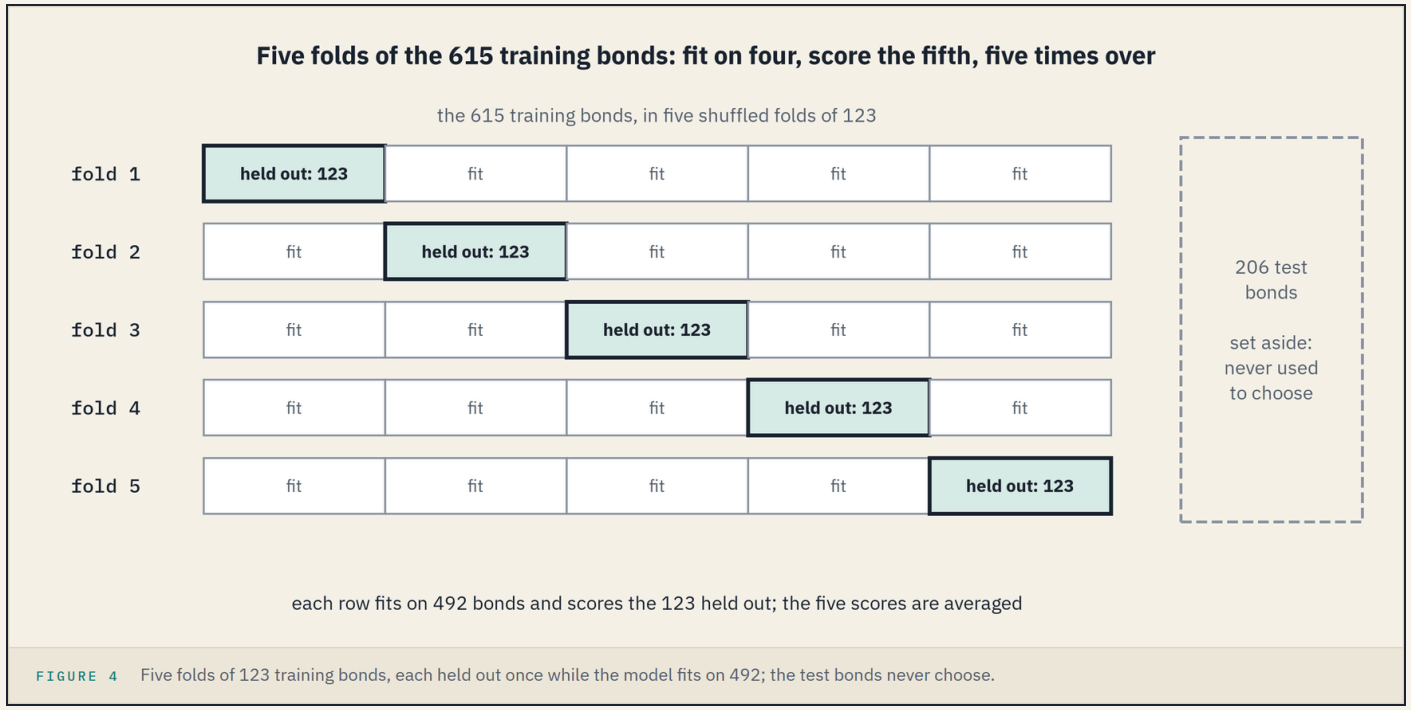


FIGURE 4 Five folds of 123 training bonds, each held out once while the model fits on 492; the test bonds never choose.

RIDGE SCORED ON FIVE FOLDS

0.001	0.897
0.003	0.897
0.010	0.897
0.032	0.897
0.100	0.897
0.316	0.896
1.000	0.885
3.162	0.850
10.000	0.783

The mean R-squared holds at 0.897 from 0.001 to 0.1, then falls as the penalty bites.

Model Performance Evaluation

THREE MODELS ON THE TEST BONDS

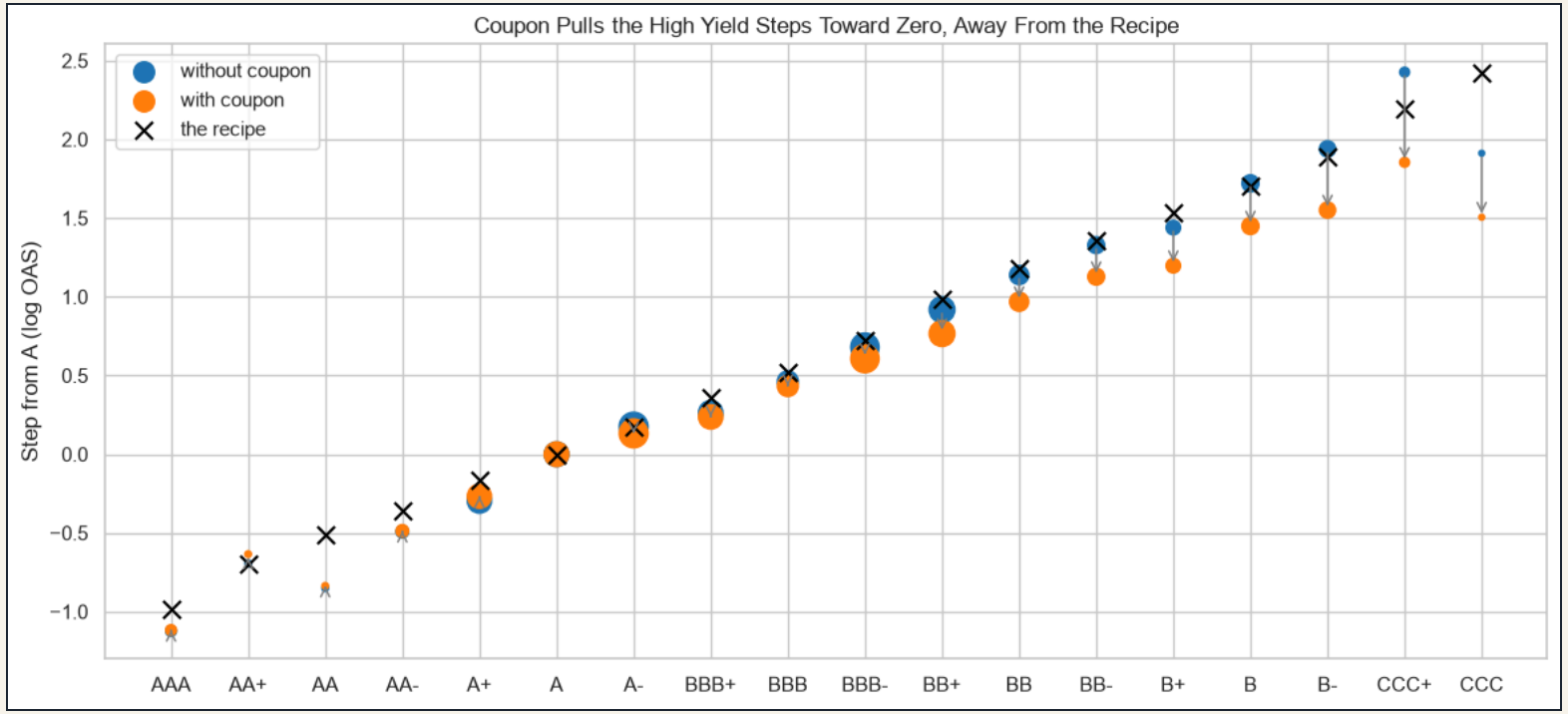
```
test_table = pd.DataFrame(index=["least squares", "ridge", "lasso"], columns=["test MAE", "test RMSE", "largest change"])
for name, model in zip(test_table.index, [ols, ridge_cv, lasso_cv]):
    fitted = model.predict(X_test)
    test_table.loc[name] = [mae_bp(y_test, fitted), rmse_bp(y_test, fitted), np.abs(model["model"].coef_ - ols_coefs).max()]
test_table
```

THE SAME SCORE ON EACH FOLD

```
bp_scorer = make_scorer(mae_bp, greater_is_better=False)
regressors = {"least squares": LinearRegression(), "ridge": RidgeCV(alphas=ridge_grid, cv=folds),
              "lasso": LassoCV(alphas=lasso_grid, cv=folds, random_state=SEED)}
fold_mae = pd.DataFrame({name: -cross_val_score(make_model(regressor), X_train, y_train, cv=folds, scoring=bp_scorer)
                         for name, regressor in regressors.items()}, index=[1, 2, 3, 4, 5])
fold_mae
```

> `greater_is_better=False` says smaller is better, so scikit-learn hands it back with a minus sign, which we flip.

Reading the Coefficients



STEPS WITHIN 0.2 OF THE RECIPE

```
with_coupon        8
without_coupon    14
dtype: int64
with_coupon      -0.230
without_coupon   -0.080
dtype: float64
Coupon and log OAS: 0.77
```

Coupon correlates 0.77 with log OAS: it takes over part of the rating's work, so the steps shrink.

The Split by Issuer

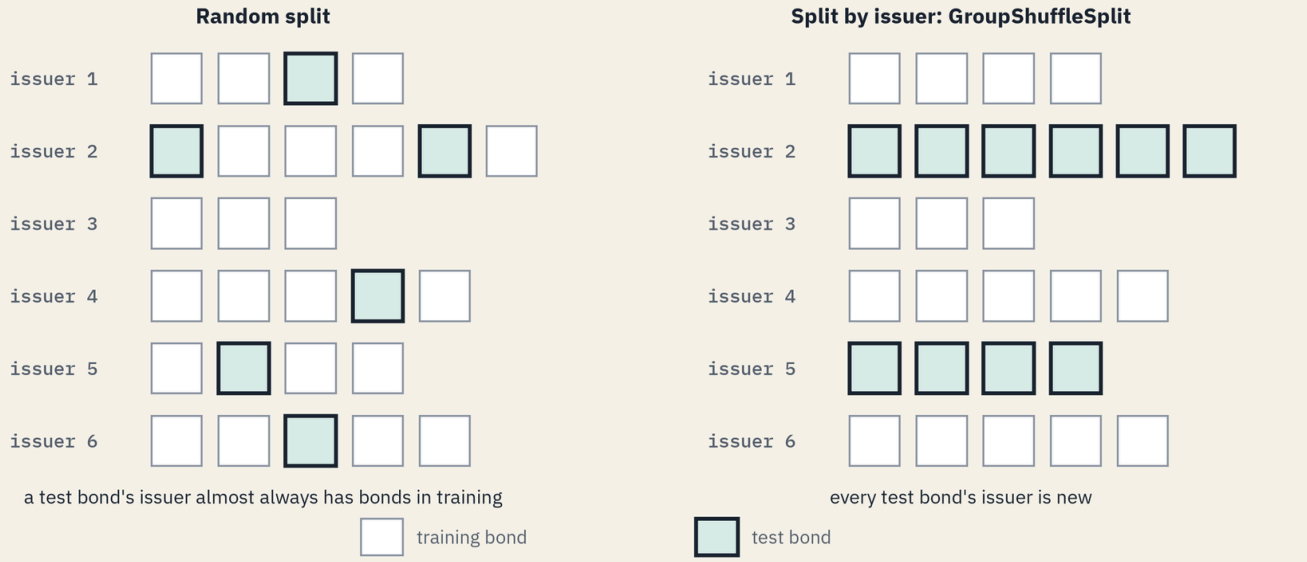


FIGURE 5 A random split scatters an issuer's bonds across both sets; a split by issuer holds whole issuers back.

HOLD BACK WHOLE ISSUERS

```
631 and 190 bonds; 38 test issuers; 0 shared
```

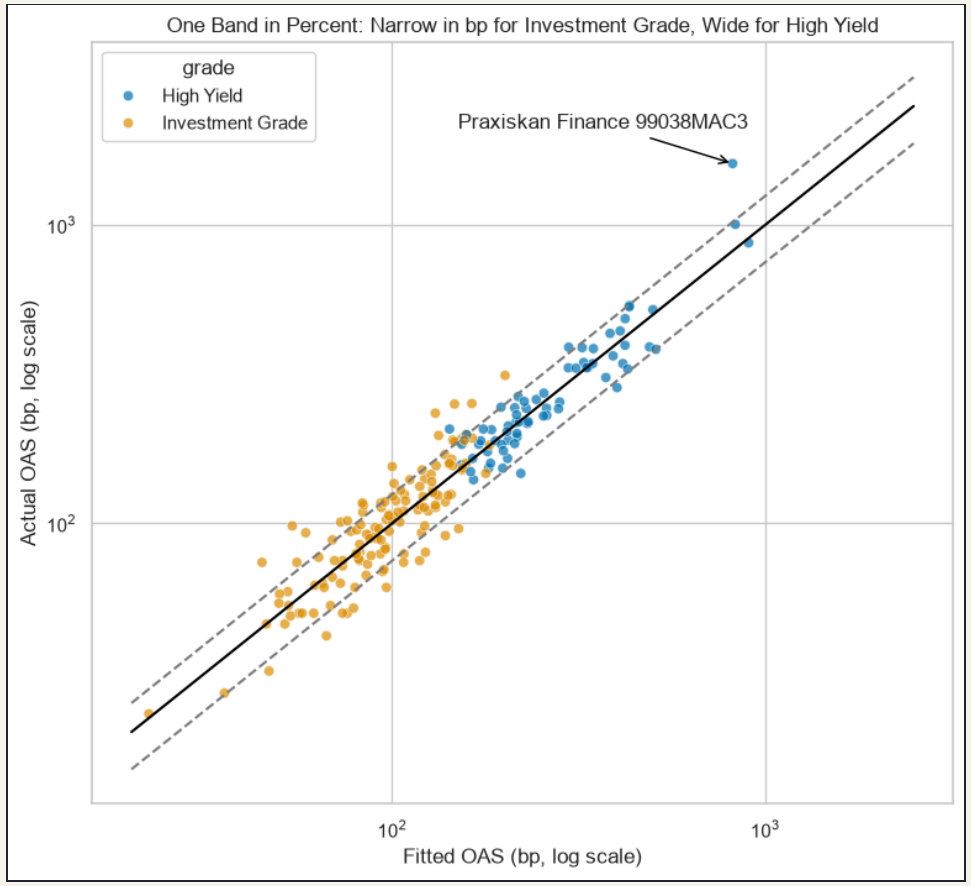
SCORE THE NEW ISSUERS

```
Test MAE, new issuers: 46.19 bp
```

Day 1 found that 203 of the 206 test bonds share an issuer with a training bond.

Find the largest misses and misses by grade.

Where the Model Fails



THE MISSES IN BP, BY GRADE

```
High Yield          47.000
Investment Grade    18.900
Within 25 percent: 0.782
Five largest misses, share of all the miss: 0.22
```

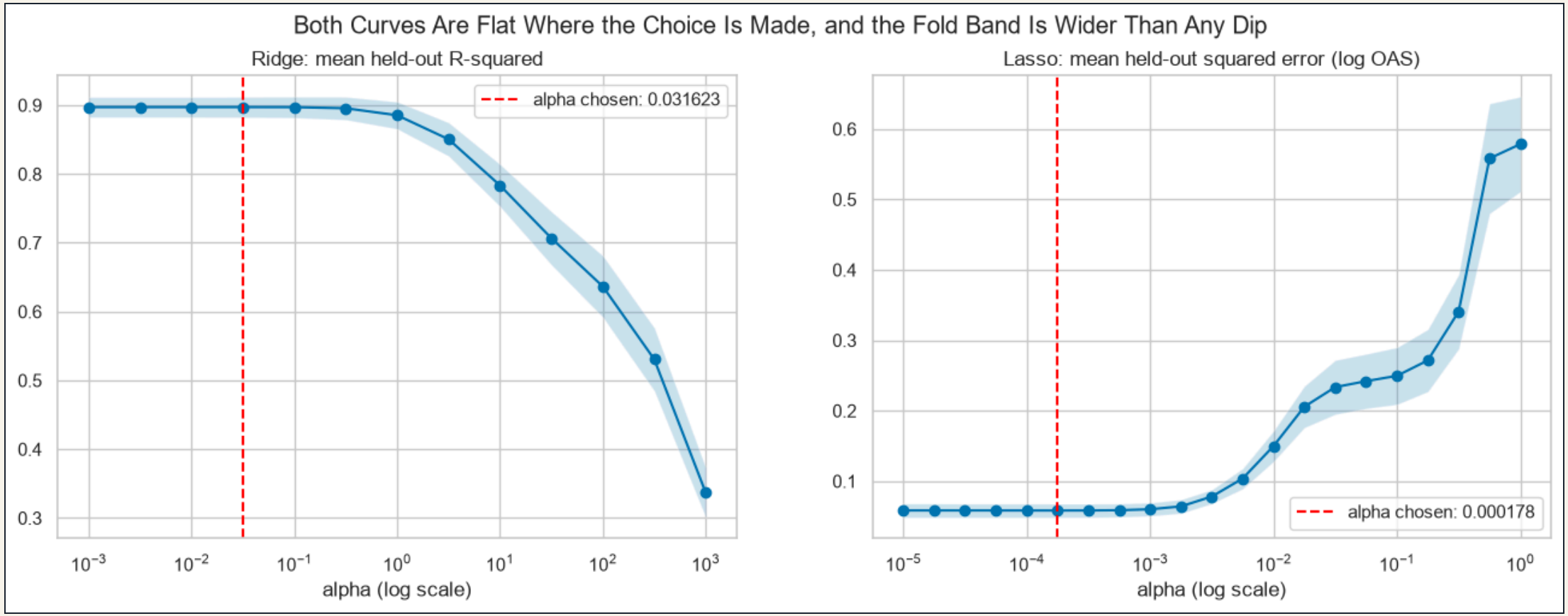
READING THE BAND

78 percent of the test bonds sit within 25 percent of their fitted spread.

A miss is the model's, not a view on any bond.

That band is 25 bp on a 100 bp bond and 250 bp on a 1,000 bp bond: high yield misses 47.0 bp on average, investment grade 18.9.

Insights and Conclusion



Week 1 ends with a model that describes these bonds and an honest price for a new issuer. Week 2 asks whether a borrower pays.

You Can Now

- Build a **`Pipeline`** with a **`ColumnTransformer`** that encodes and scales on training rows only.
- Fit ridge and lasso, read a coefficient path, and say what alpha does.
- Choose alpha by cross-validation with **`RidgeCV`** and **`LassoCV`**, and spot a choice at a grid's edge.
- Score in bp on every fold with **`make_scorer`**.
- Split by issuer with **`GroupShuffleSplit`** and **`GroupKFold`**.

NEXT

WEEK 2 · DAY 1

Logistic Regression and Odds

Week 2 brings a new file and a new question: whether a borrower pays.